



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

Semester 2

Session 2025/2026

SECJ3483 - SECTION 01
WEB TECHNOLOGY

Group Lab Activity: Security Investigation and Fix

Project Title : Secure Person BMI Full-Stack Web Application

Name	Matrics No.
MUHAMMAD AIDIL HAIKAL BIN MAZALAN	A23CS5010
MUHAMMAD AIMAN DANISH BIN MUHAMMAD EKHSAN	A23CS0119
MUHAMMAD FAKHRUL RAZZI BIN MD NOOR	A23CS0128
MUHAMMAD RIFQI BIN RAZALI	A23CS0136

2. Security investigation summary

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Data is accepted and stored in the database.	Yes	Missing backend validation because frontend validation can be bypassed using ThunderClient.
2	Access another user's BMI record	Backend should deny access	The private BMI record data is returned.	Yes	Broken Object Level Authorization because data ownership is not verified after a user logs in.
3	Access staff route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because there are no server-side role checks for staff features.
4	Access admin route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because the route lacks server-side admin protection middleware.
5	Submit XSS payload in notes	Payload should display as text	The browser executes the script by showing the alert.	Yes	Stored Cross-Site Scripting because the frontend renders input unsafely using v-html.
6	Check API response	Password/hash should not be visible	The password value is visible in the payload.	Yes	Sensitive data exposure because the API blindly returns all columns including password_hash.
7	Try SQL Injection input	Input should be treated as data	The structural database query behaves unexpectedly.	Yes	SQL Injection because the application constructs database queries using unsafe string concatenation.
8	Try to update protected fields	Backend should reject or ignore protected fields	The fields (e.g., role, user_id) are accepted and modified	Yes	Mass assignment because the backend blindly executes updates on protected fields like user_id and role.

Test 1: Add BMI with invalid data

- Did the backend accept the data?

Yes, it saved the negative and empty values directly into the database.

- Did the frontend validation stop you?

No, because I can bypass the frontend UI using Thunder Client.

- Can the same request be sent using Postman?

Yes, anyone can intercept and send raw requests to the API.

- Where should the validation be enforced?

Backend, because frontend validation can be bypassed.

The screenshot shows the Thunder Client interface with a POST request to `http://localhost:8080/api/persons`. The request body is a JSON object with the following content:

```
1 {
2   "name": "",
3   "age": -5,
4   "height": 0,
5   "weight": -70,
6   "notes": "testing invalid BMI"
7 }
```

The response status is **201 Created** with a size of **691 Bytes** and a time of **44 ms**. The response body is a JSON object with the following content:

```
1 {
2   "message": "BMI record created. This route trusts frontend data.",
3   "person": {
4     "id": 22,
5     "user_id": 1,
6     "name": "",
7     "age": -5,
8     "height": "0.00",
9     "weight": "-70.00",
10    "bmi": "0.00",
11    "category": "",
12    "notes": "testing invalid BMI",
13    "created_at": "2026-06-26 11:08:22"
14  },
15  "debug_received_body": {
16    "name": "",
17    "age": -5,
18    "height": 0,
19    "weight": -70,
20    "notes": "testing invalid BMI"
21  },
22  "debug_sql": "INSERT INTO persons (user_id, name, age, height, weight, bmi, category, notes)\nVALUES (1, '', -5, 0, -70, 0, '', 'testing invalid BMI')",
23 }
```

Test 2: Access another user's BMI record

- Did the backend return another user's BMI record?

Yes, changing the ID in the URL leaked other people's data.

- Is login alone enough?

No, it is just authentication but no authorization.

- What should the backend compare?

It must match the `user_id` from the JWT with the `user_id` on the database.

- Should staff and admin have different access?

Yes, staff and admin roles need to have different access to monitor all records.

GET `http://localhost:8080/api/persons/1` Send

Status: 200 OK Size: 435 Bytes Time: 46 ms

Response

```
1 {
2   "message": "Record returned without ownership check.",
3   "person": {
4     "id": 1,
5     "user_id": 1,
6     "name": "Muhammad Aiman Hakimi",
7     "age": 21,
8     "height": "1.70",
9     "weight": "65.00",
10    "bmi": "22.49",
11    "category": "Normal",
12    "notes": "Student sample BMI record",
13    "created_at": "2026-06-22 11:28:30"
14  },
15  "debug_sql": "SELECT * FROM persons WHERE id = 1"
16 }
```

GET `http://localhost:8080/api/persons/3` Send

Status: 200 OK Size: 435 Bytes Time: 50 ms

Response

```
1 {
2   "message": "Record returned without ownership check.",
3   "person": {
4     "id": 3,
5     "user_id": 3,
6     "name": "Ahmad Farhan Roslan",
7     "age": 21,
8     "height": "1.75",
9     "weight": "82.00",
10    "bmi": "26.78",
11    "category": "Overweight",
12    "notes": "Needs weight monitoring",
13    "created_at": "2026-06-22 11:28:30"
14  },
15  "debug_sql": "SELECT * FROM persons WHERE id = 3"
16 }
```

Test 3: Access staff route as normal user

- Did the backend block the request?

No, a normal user token could access the restricted endpoints.

- Which roles should be allowed?

Only users with a role value of 'staff' or 'admin'.

- Should checking be done in frontend route guard only?

No, attackers don't use the frontend browser to make API requests.

- Where must the final decision happen?

In the backend routing middleware.

The screenshot shows a web client interface with a request and response view. The request is a GET to `http://localhost:8080/api/staff/persons` with an Authorization header. The response is a 200 OK with a JSON body containing an error message and a list of three person records.

Request:

- Method: GET
- URL: `http://localhost:8080/api/staff/persons`
- Headers:
 - Accept: `/*`
 - User-Agent: Thunder Client (https://www.thunderclient.com)
 - Authorization: `eyJ1c2VyX2lkjoxLCjyb2xljoidXNlcilsmVtYl`
 - Content-Type: `application/json`
 - header: value

Response:

- Status: 200 OK
- Size: 9.7 KB
- Time: 22 ms
- Body (JSON):

```
{  "message": "All BMI records returned without staff role check.",  "persons": [    {      "id": 22,      "user_id": 1,      "name": "",      "age": -5,      "height": "0.00",      "weight": "-70.00",      "bmi": "0.00",      "category": "",      "notes": "testing invalid BMI",      "created_at": "2026-06-26 11:08:22",      "owner_email": "aiman@student.utm.my",      "owner_role": "user"    },    {      "id": 21,      "user_id": 1,      "name": "",      "age": -5,      "height": "0.00",      "weight": "-70.00",      "bmi": "999.00",      "category": "Normal",      "notes": "invalid test",      "created_at": "2026-06-26 10:42:47",      "owner_email": "aiman@student.utm.my",      "owner_role": "user"    },    {      "id": 20,      "user_id": 20,      "name": "Mazlina Ahmad",      "age": 45,      "height": "1.60",      "weight": "58.00"    }  ]  }
```

Test 4: Access admin route as normal user

- Did the backend accept user_id?

Yes.

- Did the backend accept role?

Yes.

- Did the backend accept bmi and category from the frontend?

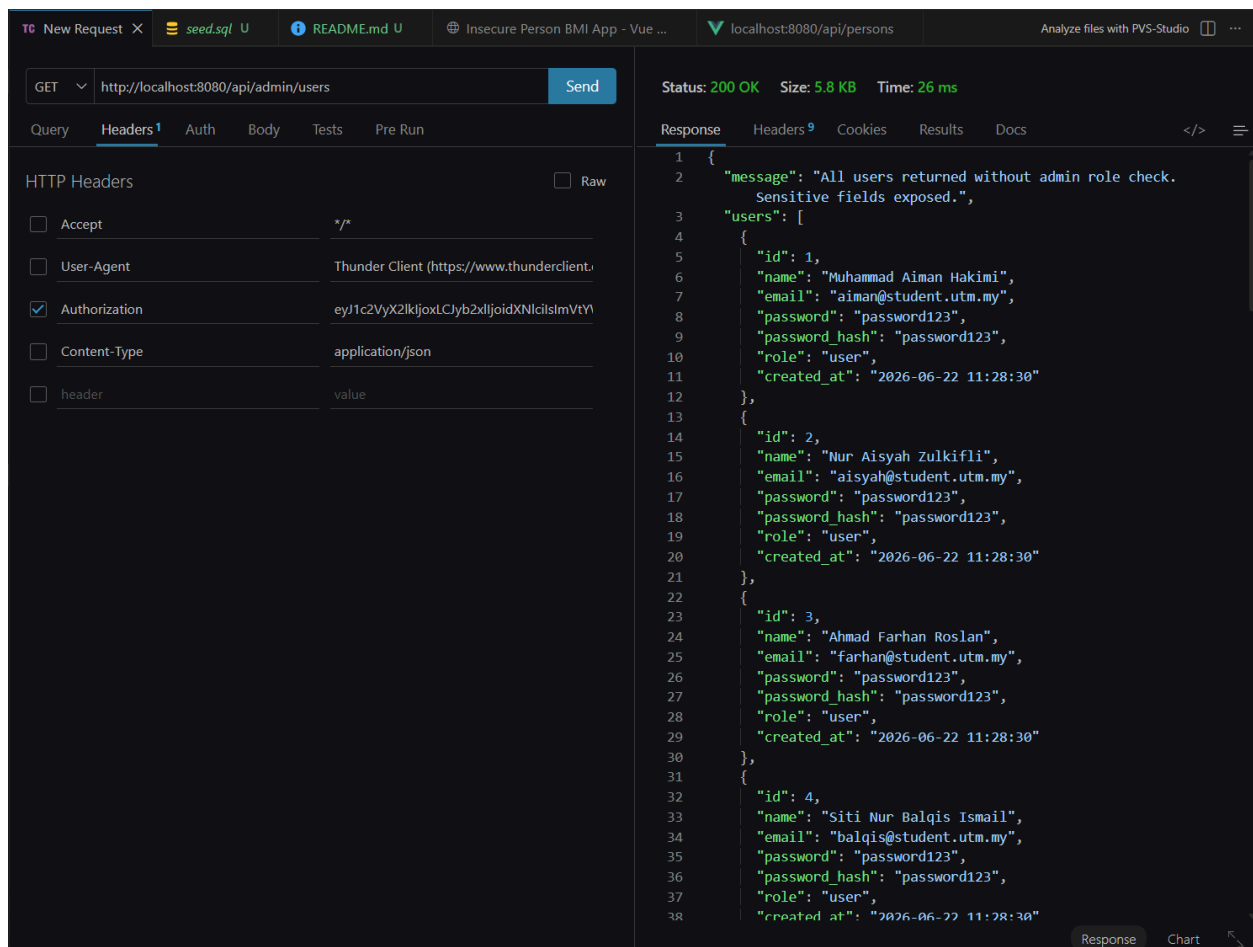
Yes, the database blindly updated them all.

- Which fields should be allowed?

Only name, age, height, weight, and notes.

- Which fields should be calculated or controlled by backend?

Structural fields like user_id and role, plus calculations like bmi and category.



The screenshot shows an HTTP client interface with the following details:

- Request:** GET `http://localhost:8080/api/admin/users`
- Status:** 200 OK, Size: 5.8 KB, Time: 26 ms
- Response (JSON):**

```
{
  "message": "All users returned without admin role check. Sensitive fields exposed.",
  "users": [
    {
      "id": 1,
      "name": "Muhammad Aiman Hakimi",
      "email": "aiman@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 2,
      "name": "Nur Aisyah Zulkifli",
      "email": "aisyah@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 3,
      "name": "Ahmad Farhan Roslan",
      "email": "farhan@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 4,
      "name": "Siti Nur Balqis Ismail",
      "email": "balqis@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    }
  ]
}
```

Test 5: Submit XSS payload in notes

- Does the API return password_hash?

Yes, it was fully visible in the JSON response payload.

- Does the frontend really need this data?

No, the client UI never needs to know password hashes.

- What fields should be returned?

Only safe profile data: id, name, email, and role.

- How can SQL SELECT be improved?

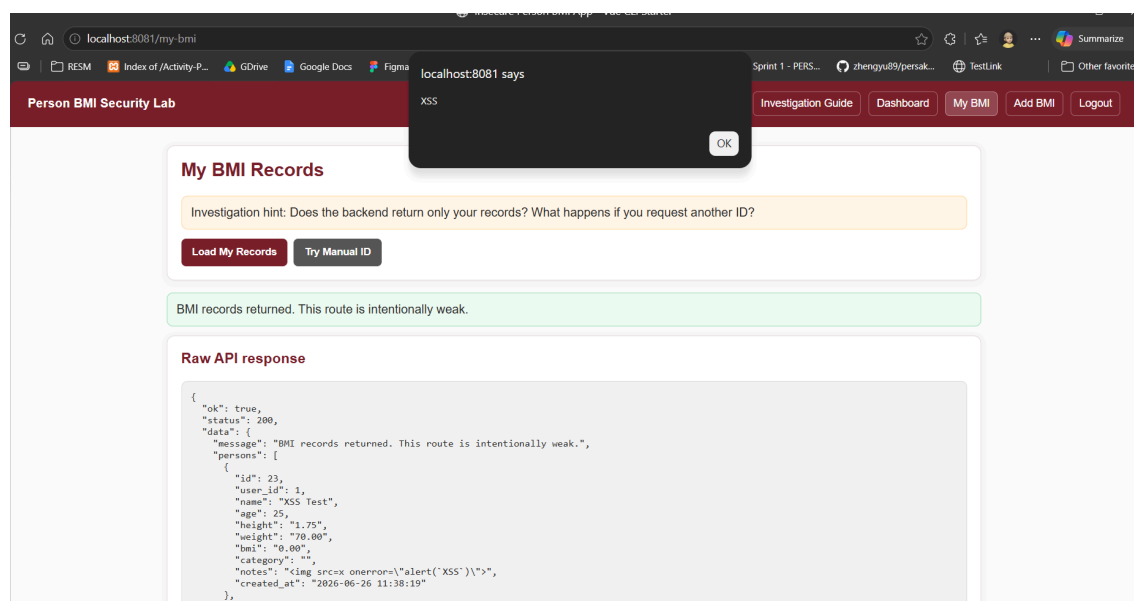
Stop using SELECT * and explicitly write out the safe columns.

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/persons`. The request body is a JSON object with the following content:

```
1 {
2   "name": "XSS Test",
3   "age": 25,
4   "height": 1.75,
5   "weight": 70,
6   "notes": "<img src=x onerror='alert('XSS')'\>"
7 }
```

The response status is 201 Created, with a size of 769 Bytes and a time of 47 ms. The response body is a JSON object with the following content:

```
1 {
2   "message": "BMI record created. This route trusts frontend data.",
3   "person": {
4     "id": 23,
5     "user_id": 1,
6     "name": "XSS Test",
7     "age": 25,
8     "height": 1.75,
9     "weight": 70.00,
10    "bmi": 0.00,
11    "category": "",
12    "notes": "<img src=x onerror='alert('XSS')'\>",
13    "created_at": "2026-06-26 11:38:19"
14  },
15  "debug_received_body": {
16    "name": "XSS Test",
17    "age": 25,
18    "height": 1.75,
19    "weight": 70,
20    "notes": "<img src=x onerror='alert('XSS')'\>"
21  },
22  "debug_sql": "INSERT INTO persons (user_id, name, age, height, weight, bmi, category, notes)\n                VALUES (1, 'XSS Test', 25, 1.75, 70, 0, '', '<img src=x onerror='alert('XSS')'\>')\n"
23 }
```



Test 6: Check API response

- Did the alert execute?

Yes, the script payload popped up in the browser window.

- Is the frontend using innerHTML or v-html?

Yes, it used unsafe binding instead of text escaping.

- What is safer for displaying normal text?

Double curly braces {{ }} text interpolation or textContent.

- Should user-generated content be rendered as HTML?

No, never render raw user input as live HTML.

The screenshot shows a web client interface with the following details:

- Request:** GET http://localhost:8080/api/admin/users
- Status:** 200 OK, Size: 5.8 KB, Time: 39 ms
- Response (JSON):**

```
{
  "message": "All users returned without admin role check. Sensitive fields exposed.",
  "users": [
    {
      "id": 1,
      "name": "Muhammad Aiman Hakimi",
      "email": "aiman@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 2,
      "name": "Nur Aisyah Zulkifli",
      "email": "aisyah@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 3,
      "name": "Ahmad Farhan Roslan",
      "email": "farhan@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    },
    {
      "id": 4,
      "name": "Siti Nur Balqis Ismail",
      "email": "balqis@student.utm.my",
      "password": "password123",
      "password_hash": "password123",
      "role": "user",
      "created_at": "2026-06-22 11:28:30"
    }
  ]
}
```


Test 7: Try SQL Injection input

- Did the SQL query behave unexpectedly?

Yes, inputting quotes broke out of the string and changed the logic.

- Was login bypassed?

Yes, the payload successfully manipulated the query parameters.

- Was extra data returned?

Yes, it fetched records without validating real credentials.

- Is the SQL query using string concatenation?

Yes, it directly glued variables into the raw query string.

The screenshot shows a web browser's developer tools with the 'Network' tab selected. A POST request to `http://localhost:8080/api/login` is shown. The request body is a JSON object: `{ "email": "aiman@student.utm.my' -- ", "password": "blablaba" }`. The response status is 200 OK, with a size of 735 Bytes and a time of 49 ms. The response body is a JSON object containing a success message, a token, user details, and debug information. The debug information shows the raw SQL query that was executed: `"SELECT * FROM users WHERE email = 'aiman@student.utm.my' -- ' AND password = 'blablaba' LIMIT 1"`. This confirms that the SQL query was using string concatenation and that the injection was successful.

```
POST http://localhost:8080/api/login
{
  "email": "aiman@student.utm.my' -- ",
  "password": "blablaba"
}
```

```
{
  "message": "Login successful. This token is intentionally insecure.",
  "token": "eyJ1c2VyX2lkIjoxLCJyb2x1IjoidXNlciIsImVtYWlsIjoiYVltYV5Ac3R1ZGVudC51dG0ubXkiLCJub3RlIjoSU5STRUNVUKVfrkFLRV9UT0tFTl9OT19TSudoQVRVUkVfIk9frVhQSVZlIn0=",
  "user": {
    "id": 1,
    "name": "Muhammad Aiman Hakimi",
    "email": "aiman@student.utm.my",
    "password": "password123",
    "password_hash": "password123",
    "role": "user",
    "created_at": "2026-06-22 11:28:30"
  },
  "debug_received_body": {
    "email": "aiman@student.utm.my' -- ",
    "password": "blablaba"
  },
  "debug_sql": "SELECT * FROM users WHERE email = 'aiman@student.utm.my' -- ' AND password = 'blablaba' LIMIT 1"
}
```

Test 8: Try to update protected fields

The screenshot shows a REST client interface with a PUT request to `http://localhost:8080/api/persons/1`. The request body is a JSON object with the following fields: `name` (Ali Updated), `age` (22), `height` (1.70), `weight` (65), `user_id` (2), `role` (admin), `bmi` (10), and `category` (Normal). The response status is 200 OK, with a size of 769 Bytes and a time of 57 ms. The response body is a JSON object with the following fields: `message` (BMI record updated. This route allows unsafe field updates.), `person` (object with the same fields as the request), `debug_received_body` (object with the same fields as the request), and `debug_sql` (UPDATE persons SET user_id = 2, name = 'Ali Updated', age = 22, height = 1.7, weight = 65, bmi = 10, category = 'Normal' WHERE id = 1).

PUT `http://localhost:8080/api/persons/1` Send

Query Headers 2 Auth **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "Ali Updated",
3   "age": 22,
4   "height": 1.70,
5   "weight": 65,
6   "user_id": 2,
7   "role": "admin",
8   "bmi": 10,
9   "category": "Normal"
10 }
```

Status: 200 OK Size: 769 Bytes Time: 57 ms

Response Headers 9 Cookies Results Docs

```
1 {
2   "message": "BMI record updated. This route allows unsafe field
3     updates.",
4   "person": {
5     "id": 1,
6     "user_id": 2,
7     "name": "Ali Updated",
8     "age": 22,
9     "height": "1.70",
10    "weight": "65.00",
11    "bmi": "10.00",
12    "category": "Normal",
13    "notes": "Student sample BMI record",
14    "created_at": "2026-06-22 11:28:30"
15  },
16  "debug_received_body": {
17    "name": "Ali Updated",
18    "age": 22,
19    "height": 1.7,
20    "weight": 65,
21    "user_id": 2,
22    "role": "admin",
23    "bmi": 10,
24    "category": "Normal"
25  },
26  "debug_sql": "UPDATE persons SET user_id = 2, name = 'Ali Updated'
27    , age = 22, height = 1.7, weight = 65, bmi = 10, category =
28    'Normal' WHERE id = 1"
29 }
```